

PONTIFICIA UNIVERSIDAD JAVERIANA
Facultad de Ingeniería – Departamento de Ingeniería de Sistemas

PLAN DE ADMINISTRACIÓN DEL PROYECTO DE SOFTWARE (SPMP)

Generación de Escenarios de Movilidad Urbana para la Toma de Decisiones en Localidades de Bogotá

Estándar base: IEEE Std 1058–1998 / ISO / IEC 12207:2008

Samuel Esteban Campos
Erick Salazar Suárez
Santiago Chitiva Contreras
Felipe Andrés Garrido Flores

Director: Ing. Andrés Oswaldo Calderón Romero, PhD.

Bogotá D.C., 13 de octubre de 2026
Versión 2.0

1. Historial de Cambios

Tabla 1: Historial de cambios del documento.

Versión	Fecha	Descripción del cambio	Autor(es)
0.1	30/04/2026	Creación inicial del documento. Definición de estructura y alcance preliminar del proyecto.	Santiago Chitiva, Samuel Campos
0.2	01/05/2026	Actualización del propósito del proyecto tras reunión con el director.	Felipe Garrido, Erick Salazar
0.3	04/05/2026	Incorporación del modelo de ciclo de vida híbrido (CRISP-DM + Scrum). Justificación de alternativas descartadas.	Samuel Campos, Santiago Chitiva
0.4	05/05/2026	Incorporación de comentarios de la Feria de Pósters. Ajuste de estimaciones Fase 3. Confirmación del alcance con director.	Felipe Garrido, Erick Salazar
0.5	06/05/2026	Revisión y aprobación por el director. Ajustes al plan de riesgos (R1 y R3 reclasificados como críticos).	Santiago Chitiva, Samuel Campos
0.6	09/05/2026	Actualización de la calendarización a 10 semanas con incorporación de ruta crítica y prerrequisitos.	Todos
1.0	10/05/2026	Versión final para entrega formal. Revisión de consistencia interna entre secciones.	Erick Salazar, Felipe Garrido
1.1	31/05/2026	Reincorporación formal del módulo web al alcance del proyecto por indicación del director y la profesora. Actualización de secciones 6.1, 6.2, 6.3, 6.5, WBS (F3.5), presupuesto y criterios de aceptación. Eliminación de restricción que excluyó la interfaz web en la versión anterior.	Todos
1.2	02/06/2026	Correcciones derivadas de las sugerencias del Profesor: adición de paginación, renumeración de figuras, ajuste de figuras a márgenes, inclusión de captions en figuras y tablas, y verificación de referencias citadas en el texto.	Todos
2.0	13/10/2026	Migración del documento a Typst (fuente versionada en `docs/spmp/`). Actualización al estado del proyecto (rama `feat/fases-b-f-estilos-cierres`): • alcance ampliado con la demanda parametrizable, el módulo de intersecciones y semaforización y el simulador 3D comparativo (6.1, 6.2, 6.4, 6.5, 11); • herramientas reales del sistema (8.2), criterios de aceptación (8.3), WBS y calendarización con los Sprints 6 a 8 (9.3); • riesgos R8 a R11 (12.2), ítems de configuración y estructura del repositorio (12.3, Anexo A) y nueva métrica de pruebas automatizadas (12.4).	Equipo EFSS

2. Prefacio

El presente documento constituye el Plan de Administración del Proyecto de Software (SPMP) para el proyecto de grado «Generación de Escenarios de Movilidad Urbana para la Toma de Decisiones en Localidades de Bogotá», desarrollado por cuatro estudiantes de Ingeniería de Sistemas de la Pontificia Universidad Javeriana.

El proyecto tiene como propósito diseñar e implementar dos módulos:

- un módulo de procesamiento de datos que automatiza la transformación de la información de la Encuesta de Movilidad de Bogotá (2023) y de los aforos vehiculares en hora de máxima demanda en archivos de entrada válidos para el simulador de tráfico SUMO (Simulation of Urban MObility) [1];
- un módulo web que permite visualizar los resultados producidos por SUMO y parametrizar nuevas simulaciones desde una interfaz gráfica accesible, sin interactuar con archivos de configuración técnicos.

En la versión 2.0, el módulo web incorpora además el análisis de intersecciones y un simulador 3D que compara, lado a lado, un cruce sin semáforo y el mismo cruce con un semáforo propuesto.

Este documento está dirigido principalmente al equipo de desarrollo, al director del proyecto y a los jurados evaluadores del trabajo de grado. El SPMP se basa en el estándar IEEE Std 1058-1998 [2], complementado con ISO/IEC 12207:2008 [3]. Los procesos de soporte se describen con diagramas BPMN 2.0 [4], cuya ubicación se señala en cada sección correspondiente.

3. Tabla de Contenidos

1. Historial de Cambios

. 1

2. Prefacio

. 2

6 Vista General del Proyecto 5
- 6.1 Visión del Producto 5
- 6.2 Propósito, Alcance y Objetivos 5
- 6.3 Supuestos y Restricciones 6
- 6.4 Entregables 7
- 6.5 Resumen de Calendarización y Presupuesto 8
- 6.6 Evolución del Plan 9

7 Glosario 9

8 Contexto del Proyecto 10
- 8.1 Modelo de Ciclo de Vida 10
- 8.2 Lenguajes y Herramientas 11
- 8.3 Plan de Aceptación del Producto 14
- 8.4 Organización del Proyecto y Comunicación 17

9 Administración del Proyecto 21
- 9.1 Métodos y Herramientas de Estimación 21
- 9.2 Inicio del Proyecto 25
- 9.3 Planes de Trabajo del Proyecto 27

10 Monitoreo y Control del Proyecto 29
- 10.1 Administración de Requerimientos 29
- 10.2 Monitoreo y Control de Progreso 30
- 10.3 Cierre del Proyecto 33

11 Entrega del Producto 33

12 Procesos de Soporte 34
12.1 Ambiente de Trabajo 34
12.2 Análisis y Administración de Riesgos 34
12.3 Administración de Configuración y Documentación 36
12.4 Métricas y Proceso de Medición 38
12.5 Control de Calidad 39
13 Anexos 40
13.1 Anexo A – Estructura del Repositorio 40
13.2 Anexo B – Registro de Riesgos Actualizable 40
13.3 Anexo C – Compilación de los documentos 40
14. Referencias
..... 40

4. Lista de Figuras

Figura 1 Diagrama alto nivel del modelo de ciclo de vida. 10
Figura 2 Diagrama BPMN – Modelo de ciclo de vida híbrido CRISP-DM / Scrum. 11
Figura 3 Organigrama del equipo de desarrollo. 18
Figura 4 Diagrama WBS (línea base v1.x). 23
Figura 5 Diagrama Gantt (línea base v1.x). 23
Figura 6 Diagrama BPMN – Proceso de administración de requerimientos. 30
Figura 7 Diagrama BPMN – Proceso de identificación y seguimiento de riesgos. 34
Figura 8 Diagrama BPMN – Proceso de control de configuración. 36
Figura 9 Diagrama BPMN – Proceso de control de calidad. 39

5. Lista de Tablas

Tabla 1 Historial de cambios del documento. 1
Tabla 2 Entregables del proyecto. 7
Tabla 3 Hitos del proyecto. 8
Tabla 4 Resumen de presupuesto (horas-persona). 8
Tabla 5 Glosario. 9
Tabla 6 Herramientas y lenguajes del proyecto. 11
Tabla 7 Criterios de aceptación de entregables. 14
Tabla 8 Entidades externas e interfaces. 17
Tabla 9 Roles del equipo y responsabilidades. 18
Tabla 10 Complejidad técnica por componente. 22
Tabla 11 Asignación de recursos por fase. 24
Tabla 12 Entrenamiento del personal. 25
Tabla 13 Infraestructura y configuración. 25
Tabla 14 Riesgos de infraestructura. 27
Tabla 15 Calendarización con camino crítico (WBS + prerrequisitos). 28
Tabla 16 Asignación de responsabilidades por fase e integrante. 29
Tabla 17 Métricas del proyecto. 30
Tabla 18 Registro de riesgos identificados. 35
Tabla 19 Plan de mitigación de riesgos prioritarios. 35
Tabla 20 Ítems de configuración. 37
Tabla 21 Ciclo de vida de artefactos. 37
Tabla 22 Métricas y proceso de medición. 38
Tabla 23 Procesos de control de calidad. 39

Tabla 24 Estructura del repositorio. 40

6 Vista General del Proyecto

6.1 Visión del Producto

Bogotá enfrenta uno de los problemas de congestión vehicular más severos del mundo: según el TomTom Traffic Index 2025, ocupa el séptimo lugar global con un índice del 69,9% y velocidades promedio en horas pico que apenas superan los 15 km/h. Pese a la existencia de herramientas de simulación de tráfico de clase mundial como SUMO [1], su uso por parte de planificadores y analistas de movilidad está limitado por la alta complejidad técnica que implica transformar datos reales de movilidad en los formatos de entrada que el simulador requiere.

La visión del producto es reducir esa barrera con un módulo de procesamiento de datos que, a partir de la Encuesta de Movilidad de Bogotá 2023 [5] y datos de aforos vehiculares, genere automáticamente los archivos necesarios para ejecutar simulaciones en SUMO. El sistema permite a investigadores y analistas explorar escenarios hipotéticos de movilidad sin necesidad de intervención manual en la preparación de datos.

El alcance de la versión 1 del proyecto está delimitado a la localidad de Usaquén como caso piloto, con un diseño orientado a escalar a todas las localidades de Bogotá en versiones futuras. El sistema comprende dos módulos: un módulo de procesamiento de datos (pipeline ETL) y un módulo web para la visualización interactiva de resultados y la parametrización de nuevas simulaciones, ambos incluidos en el alcance actual por instrucción del director del proyecto.

Versión 2.0. Con el pipeline y el visor operativos, el producto avanza hacia el análisis de **control de tráfico** previsto en el Sprint 4. Se agregan:

- un módulo de intersecciones con ficha, edición de programas, Webster, recorte de zonas, comparación de programas y priorización de cruces;
- un **simulador 3D comparativo** que muestra en pantalla partida un cruce sin semáforo y el mismo cruce con un semáforo propuesto.

El simulador usa la red 3D real de Usaquén, los edificios y la demanda de la encuesta. Permite seguir en tercera persona a un vehículo o peatón y ver en pantalla su velocidad, su nivel de riesgo y las decisiones que toma, para hacer tangible el efecto de la intervención antes de implementarla.

6.2 Propósito, Alcance y Objetivos

Propósito. El proyecto busca eliminar la brecha entre los datos disponibles de movilidad urbana en Bogotá y las herramientas de simulación de tráfico. La transformación de datos de la Encuesta de Movilidad en insumos para SUMO requiere actualmente procesos manuales que toman horas de trabajo especializado. El módulo desarrollado automatiza este proceso de extremo a extremo.

Alcance – Incluye

- Análisis y comparación de fuentes de datos de movilidad disponibles (Encuesta de Movilidad 2023, aforos vehiculares HMD, OpenStreetMap).
- Desarrollo del pipeline ETL que transforma los datos en archivos de entrada de SUMO: red vial (`.net.xml`), zonas TAZ (`.taz.xml`), viajes y rutas (`.trips.xml`, `.rou.xml`) y tipos de vehículo.
- Validación del pipeline en la localidad de Usaquén, verificando coherencia con aforos reales mediante el estadístico GEH.
- Análisis de al menos dos escenarios hipotéticos de movilidad (infraestructura vial y control de tráfico).
- Desarrollo de un módulo web que permita visualizar de forma interactiva los resultados producidos por SUMO (métricas de congestión, tiempos de viaje) y parametrizar nuevas simulaciones desde una interfaz gráfica, sin necesidad de editar archivos manualmente.

- *(v2.0)* Demanda parametrizable con presets de transferencia modal, conteos por modo, semilla, estilos de conducción y cierres viales, con simulaciones precalculadas de los presets.
- *(v2.0)* Módulo de intersecciones y semaforización: catálogo de semáforos, programas, Webster, recorte de zonas, comparación de programas y priorización de cruces sin semáforo.
- *(v2.0)* Simulador 3D comparativo de semáforo propuesto, con seguimiento en tercera persona de cualquier actor, HUD de velocidad, riesgo (TTC, DRAC, PET) y decisiones, minimapa 2D y selección de vías.

Alcance – Excluye
- Integración en tiempo real con APIs de tráfico comerciales.
- Despliegue en sistemas de producción institucionales.
- Calibración del modelo para localidades distintas a Usaquén en esta versión.
- Programas semafóricos reales de la Secretaría Distrital de Movilidad (se usan los generados por netconvert a partir de OpenStreetMap).

Objetivo General. Diseñar e implementar un módulo de procesamiento de datos que automatice la transformación de fuentes de datos de movilidad disponibles para Bogotá en archivos de entrada válidos para el simulador SUMO, eliminando la intervención manual en la generación de redes viales, matrices origen-destino y archivos de rutas.

Objetivos Específicos
- Analizar y comparar las fuentes de datos de movilidad disponibles para Bogotá para determinar cuáles permiten construir matrices O/D suficientemente representativas [6].
- Desarrollar el módulo de procesamiento que automatice la transformación en archivos válidos para SUMO, con manejo de datos faltantes y resultados reproducibles.
- Validar el módulo generando insumos de simulación para la localidad de Usaquén, verificando coherencia con aforos reales y ejecutando el escenario en SUMO.
- Desarrollar un módulo web que permita visualizar de forma interactiva los resultados generados por SUMO y parametrizar nuevas simulaciones desde una interfaz gráfica, reduciendo la barrera técnica de acceso al sistema para usuarios del área de movilidad. En la versión 2.0 este objetivo incluye el análisis de intersecciones y el simulador 3D comparativo.

6.3 Supuestos y Restricciones

Supuestos
- La Encuesta de Movilidad de Bogotá 2023 y la base de datos de aforos vehiculares HMD son accesibles y de calidad suficiente para construir matrices O/D representativas.
- La red vial de Bogotá en OpenStreetMap tiene suficiente cobertura y atributos para generar una red simulable en SUMO para la localidad de Usaquén.
- Los cuatro integrantes del equipo están disponibles durante las 16 semanas del proyecto con una dedicación aproximada de 10 horas semanales por persona.
- El equipo de cómputo disponible tiene al menos 8 GB de RAM y un procesador de 4 núcleos para ejecutar SUMO a escala de localidad.
- SUMO en su versión estable (1.27, instalada con el paquete `eclipse-sumo`) soporta todos los modos de transporte a modelar.
- *(v2.0)* El navegador del usuario soporta WebGL 2 para el render 3D del simulador.

Restricciones
- El proyecto está planeado para completarse en 10 semanas de trabajo efectivo. Aunque el semestre cuenta con 16 semanas para la entrega final, la calendarización se diseñó dejando semanas adicionales de contingencia y ajuste, con el fin de disponer de tiempo suficiente para corregir errores,

realizar pruebas, atender imprevistos y efectuar mejoras antes de la entrega final. La extensión v2.0 (Sprints 6 a 8) usa ese margen (ver 9.3).
- Todas las herramientas deben ser de código abierto o de uso académico gratuito (sin licencias comerciales).
- El alcance geográfico está limitado a la localidad de Usaquén para la validación.
- El código fuente debe seguir la guía de estilo PEP 8 y estar versionado en GitHub.
- El módulo web debe ser accesible desde navegadores modernos sin instalación adicional; no se contempla despliegue en producción institucional en esta versión.

6.4 Entregables

Tabla 2: Entregables del proyecto.

Entregable	Estándar	Descripción	Aceptación	Semana
SPMP corregido	IEEE 1058-1998	Plan de administración del proyecto (fuente Typst + PDF)	Equipo / Director / Jurados	3
SRS corregido	IEEE 830-1998	Especificación de requisitos del software [7] (fuente Typst + PDF)	Equipo / Director / Jurados	4
SDD	IEEE 1016-2009	Documento de diseño del sistema [8]	Equipo / Director	6
Plan de Pruebas	ISO/IEC/IEEE 29119	Plan detallado de pruebas [9]	Equipo / Director	7
Código Fuente	PEP 8 / API TraCI	Repositorio GitHub con `docker-compose.yml` y servicio (pipeline ETL, API,	Ejecución en	9

Entregable	Estándar	Descripción	Semana
		simulador 3D) y visor web `kepler`	
Módulo de semaforización y simulador 3D *(v2.0)*	PEP 8 / API TraCI	Intersecciones priorización y comparación 3D sin / con semáforo propuesto	11–14
Informe Final	IEEE 1063-2001	Documento final con resultados	10–16

6.5 Resumen de Calendarización y Presupuesto

Hitos del Proyecto

Tabla 3: Hitos del proyecto.

Hito	Descripción	Semana
H1	Entrega SPMP y SRS corregidos. Arranque formal del proyecto	1–2
H2	Pipeline ETL funcional. Matrices O/D y red vial generadas	2–6
H3	Módulo web funcional: visualización de resultados y parametrización de simulaciones	6–8
H4	Entrega SDD y Plan de Pruebas	8–10
H5	Escenario base validado en SUMO (Sprint 1)	10–12
H6	Escenarios hipotéticos analizados (Sprint 3–4)	12–14
H7	Entrega informe final	14–16
H8 *(v2.0)*	Módulo de intersecciones y simulador 3D comparativo operativos (Sprints 6 y 7)	11–14

Resumen de Presupuesto

Tabla 4: Resumen de presupuesto (horas-persona).

Ítem	Descripción	Cantidad	Costo equivalente
Horas-persona totales	Línea base: 4 personas × 10 semanas × 14 h/semana + 40 h del Sprint 5. Extensión v2.0: Sprints 6 a 8 (120 h)	720 h	—

Ítem	Descripción	Cantidad	Costo equivalente
Herramientas de software	SUMO, Python, FastAPI, React, kepler.gl, three.js, GitHub, GitHub Actions, Typst (todos gratuitos)	—	$0 COP
Datos de entrada	Encuesta Movilidad 2023, aforos, OSM, Microsoft Building Footprints (todos públicos)	—	$0 COP
Infraestructura de cómputo	Equipos personales del equipo + Google Colab	—	$0 COP
Total monetario del proyecto			$0 COP

El proyecto no maneja presupuesto monetario. Todas las herramientas son de código abierto y los datos de entrada son de acceso público. El costo total equivalente se expresa en horas-persona: la línea base es de 600 h-p (560 h-p de 4 integrantes × 10 semanas × 14 h/semana, más 40 h del módulo web). La extensión v2.0 suma 120 h-p estimadas para los Sprints 6 a 8, para un total de 720 h-p.

6.6 Evolución del Plan

El presente documento es un artefacto vivo que se actualizará en cada entrega formal del proyecto. Los cambios al plan se gestionan a través del proceso de control de configuración descrito en la sección 12.3. Cualquier modificación sustancial al alcance, calendario o recursos debe ser aprobada por el director y registrada en el historial de cambios (sección 1). La versión 2.0 (ampliación del alcance con semaforización y simulador 3D) se somete a esa aprobación.

7 Glosario

Tabla 5: Glosario.

Término / Acrónimo	Definición
SUMO	Simulation of Urban MObility. Simulador de tráfico microscópico de código abierto.
FastAPI	Framework de Python para construir APIs web (REST y WebSocket). Base del backend del módulo web.
ETL	Extract, Transform, Load. Pipeline de procesamiento de datos por etapas.
O/D	Matriz Origen-Destino. Estructura que cuantifica viajes entre pares de zonas geográficas.
TAZ	Traffic Analysis Zone. Unidad espacial básica del modelo de demanda de transporte.
UPZ	Unidad de Planeamiento Zonal. División administrativa de Bogotá usada como TAZ.
HMD	Hora de Máxima Demanda. Período de mayor volumen vehicular (AM o PM).
GEH	Estadístico de validación de modelos de tráfico. Valor < 5 indica buena calibración.
RMSE	Root Mean Square Error. Métrica de error cuadrático medio usada en validación.

Término / Acrónimo	Definición
CRISP-DM	Cross-Industry Standard Process for Data Mining. Metodología iterativa de ciencia de datos.
Scrum	Marco ágil de desarrollo de software basado en sprints iterativos cortos.
TraCI	Traffic Control Interface. API de SUMO para control en tiempo real de simulaciones.
netconvert	Herramienta de SUMO para convertir redes viales de OSM al formato `.net.xml` (y, en v2.0, para instalar el semáforo propuesto).
duarouter	Herramienta de SUMO para asignar rutas a los viajes de las matrices O/D.
OSM	OpenStreetMap. Mapa vial colaborativo de código abierto.
PEP 8	Guía de estilo para código Python de la Python Software Foundation.
WBS	Work Breakdown Structure. Estructura de descomposición de tareas del proyecto.
SPMP	Software Project Management Plan. Plan de Administración del Proyecto de Software.
SRS	Software Requirements Specification. Especificación de Requisitos del Software.
SDD	Software Design Document. Documento de Diseño del Software.
tlLogic	Programa de un semáforo en la red de SUMO: fases con duración y estado por conexión.
Webster	Método clásico de cálculo del ciclo óptimo de un semáforo a partir de los flujos críticos [10].
TTC / DRAC / PET	Indicadores sustitutos de seguridad [11]: tiempo a la colisión, desaceleración requerida para evitarla y tiempo entre el paso de dos actores por un mismo punto de conflicto.
Lockstep	Ejecución de dos simulaciones avanzando el mismo intervalo en cada paso, para compararlas instante a instante.
kepler.gl / three.js	Bibliotecas de visualización del visor web: mapa animado de viajes y render 3D del simulador comparativo.
Typst	Lenguaje de marcado y compilador de documentos (similar a LaTeX) usado para el SPMP, el SRS y la VFP.

8 Contexto del Proyecto

8.1 Modelo de Ciclo de Vida

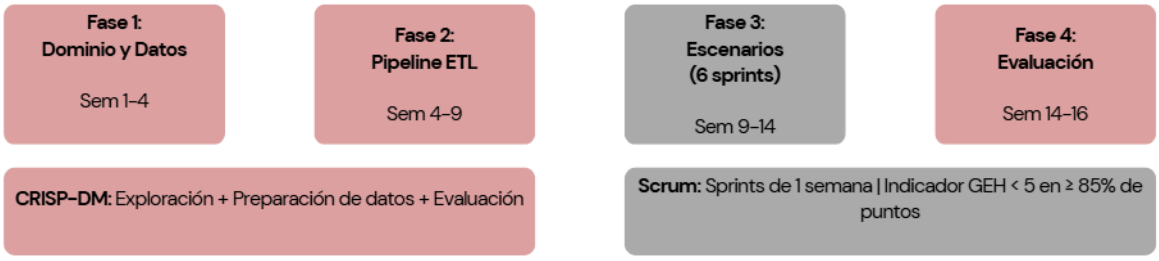


Figura 1: Diagrama alto nivel del modelo de ciclo de vida.

Descripción del Modelo. El proyecto adopta un modelo de ciclo de vida híbrido que combina CRISP-DM [12] para las fases intensivas en datos (Fases 1 y 2) con Scrum [13] para la construcción iterativa del escenario de simulación (Fase 3). La Fase 4 de evaluación retoma la estructura de CRISP-DM (véase la Figura 1).

Las Fases 1 y 2 siguen CRISP-DM porque el trabajo en esa etapa es de naturaleza exploratoria y analítica: comprender las fuentes de datos, evaluar su calidad, identificar brechas y construir el pipeline ETL. A partir de la Fase 3, el proyecto migra a Scrum con sprints semanales, dado que el trabajo pasa a ser incremental y orientado a componentes verificables de software. Además, se aplicará el tablero visual de la metodología Kanban para gestionar las tareas.

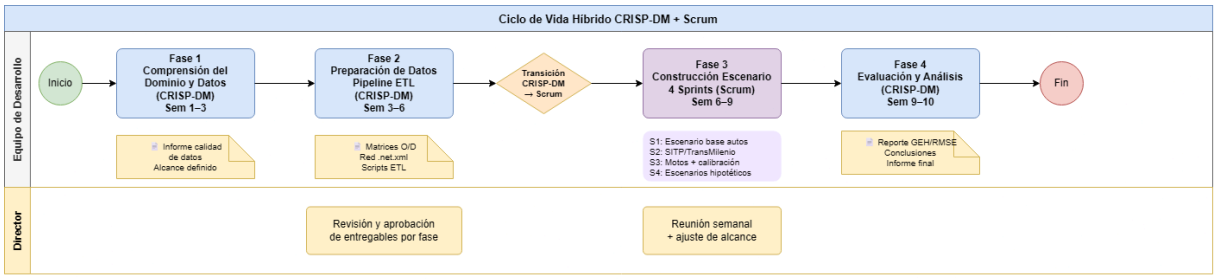


Figura 2: Diagrama BPMN – Modelo de ciclo de vida híbrido CRISP-DM / Scrum.

La Figura 2 muestra las 4 fases del proyecto, la transición entre metodologías, los roles involucrados en cada fase y los artefactos que produce cada una (matrices O/D, red `.net.xml`, código Python, reportes GEH). En la versión 2.0 la Fase 3 se extiende con los Sprints 5 a 8 (ver 9.3.1); la estructura de fases no cambia.

Análisis de Alternativas. Se evaluaron los siguientes modelos de ciclo de vida antes de seleccionar el híbrido:

- **Cascada:** descartado por su rigidez ante cambios en la calidad de los datos de entrada, que no se conoce hasta la exploración.
- **CRISP-DM puro:** insuficiente para la Fase 3, que requiere gestión de sprints y entregas incrementales de software.
- **RUP:** se descartó por su excesiva burocracia que podría frenar la agilidad del análisis.
- **Híbrido CRISP-DM + Scrum + Kanban:** modelo que combina exploración de datos iterativa con entregas incrementales de software.

Prácticas Específicas

- Control de versiones: Git + GitHub con ramas por sprint (`main`, `develop`, `feat/*`).
- Revisiones de código: revisión obligatoria de al menos un compañero antes de hacer merge a cada cambio.
- Integración continua: GitHub Actions para ejecutar pruebas unitarias automáticamente en cada commit.
- Reuniones de equipo: dos reuniones semanales (lunes: planeación/seguimiento; viernes: revisión).
- Reunión con director: una vez a la semana, al cierre de cada sprint.
- Definición de «done» para simulación: componente validado con GEH < 5 en al menos el 85% de los puntos de aforo.
- Definición de «done» para software (v2.0): pruebas automatizadas del módulo en `simulation-service/test/` en verde, `npm run build` del visor sin errores y demostración del flujo en el visor.

8.2 Lenguajes y Herramientas

Tabla 6: Herramientas y lenguajes del proyecto.

Herramienta	Propósito	Justificación
Python 3.11+	Lenguaje principal del pipeline ETL, la API y el control de SUMO	Ecosistema estándar para análisis de datos. Requerido por la API TraCI de SUMO.
FastAPI + Uvicorn	Backend REST y WebSocket del módulo web	Herramienta flexible y fácil de configurar; soporta WebSocket para el simulador en vivo.
SUMO 1.27 (`eclipse-sumo`)	Motor de simulación microscópica de tráfico	Herramienta de código abierto estándar. Única alternativa open source con soporte multimodal completo. Instalada con pip, deja `sumo`, `netconvert` y `duarouter` en el entorno virtual.
TraCI / sumolib 1.27.1	Control en vivo de SUMO y lectura de la red	Permite medir el avance real de las corridas y correr las dos simulaciones del simulador 3D en lockstep.
pandas / geopandas	Procesamiento de datos tabulares y geoespaciales	Estándar de Python para ETL de datos estructurados y geoespaciales.
OpenStreetMap + osmium	Fuente y procesamiento de la red vial	Única fuente gratuita con cobertura global y nivel de detalle suficiente.
netconvert / duarouter / cutRoutes / randomTrips	Conversión OSM a `.net.xml`, rutas, recorte de demanda y tráfico de fondo	Herramientas oficiales del ecosistema SUMO; garantizan compatibilidad.
React 18 + Vite	Visor web (`kepler/`)	Componentes reutilizables y servidor de desarrollo con proxy HTTP/ WebSocket a la API.

Herramienta	Propósito	Justificación
kepler.gl 3.1	Mapa animado de viajes y capas de semáforos y priorización	Capa de trayectorias nativa para miles de viajes.
three.js 0.169 + maplibre-gl 4.7	Render 3D del simulador y minimapa 2D	three.js da control total de la cámara en tercera persona; maplibre no requiere token para teselas OSM.
Git + GitHub	Control de versiones y repositorio del proyecto	Estándar de la industria. Permite colaboración distribuida y control de cambios.
GitHub Actions	Integración continua (CI)	Gratuito. Automatiza la ejecución de pruebas en cada commit.
pytest	Pruebas unitarias e integración	Estándar de facto para pruebas en Python. Integración nativa con GitHub Actions.
Jupyter Notebooks	Exploración y caracterización de datos (Fases 1 y 2)	Permite análisis exploratorio iterativo con visualización integrada.
QGIS	Validación geoespacial de la red vial y TAZ	Herramienta GIS de código abierto; integración con OSM y geopandas.
Typst	Documentos formales (SPMP, SRS, VFP)	Fuente de texto versionable en Git; compila a PDF en segundos.

8.2.1 Análisis de Alternativas y Justificación

- **Backend:** FastAPI frente a Flask o Django. FastAPI valida las solicitudes con Pydantic, genera la documentación en `/docs` y soporta WebSocket de forma nativa, lo que el simulador 3D necesita.
- **Visor de resultados:** kepler.gl frente a Leaflet o deck.gl directo. kepler.gl ya trae la capa de trayectorias animadas, la línea de tiempo y filtros, lo que evita reimplementarlos.
- **Render 3D:** three.js frente a deck.gl o la interfaz gráfica de SUMO (sumo-gui). deck.gl está pensado para vistas cartográficas y no ofrece una cámara de persecución; sumo-gui no se integra al navegador. three.js permite construir la escena desde la geometría de la red y controlar la cámara libremente.
- **Documentos:** Typst frente a Word o LaTeX. Es texto plano, por lo que se versiona y revisa en GitHub como el código, y es más sencillo que LaTeX.

8.3 Plan de Aceptación del Producto

Tabla 7: Criterios de aceptación de entregables.

Entregable	Criterio de aceptación	Herramienta de verificación
SPMP	Cubre todas las secciones del estándar IEEE 1058. Consistencia interna entre secciones. Aprobado por el director.	Revisión documental del director
SRS	Requerimientos trazables, no ambiguos y verificables. Matriz de trazabilidad incluida.	Revisión formal + checklist IEEE 830
SDD	Arquitectura documentada. Decisiones de diseño justificadas. Trazabilidad con SRS.	Revisión formal del director
Plan de Pruebas	Cobertura del 100% de los requerimientos funcionales del SRS.	Matriz de trazabilidad SRS-pruebas

Entregable	Criterio de aceptación	Herramienta de verificación
Código fuente	Pasa el 90% de las pruebas unitarias. Cumple PEP 8. Documentado con docstrings.	pytest + GitHub Actions + pylint
Pipeline validado	GEH < 5 en al menos el 85% de los puntos de aforo. Ejecutable en SUMO sin errores.	Métricas GEH y RMSE vs. aforos reales
Informe final	Incluye todos los elementos del IEEE 1063-2001. Aprobado por jurados.	Revisión de jurados en la exposición
Módulo web	Permite visualizar métricas de SUMO. Permite lanzar una simulación parametrizada exitosamente	Prueba funcional manual + revisión del director

Entregable	Criterio de aceptación	Herramienta de verificación
	desde la interfaz.	
Módulo de semaforización *(v2.0)*	Muestra la ficha de cualquier semáforo con volúmenes por acceso, guarda un programa válido, calcula Webster y lista los cruces prioritarios sin semáforo.	Pruebas automatizadas + demostración
Simulador 3D comparativo *(v2.0)*	Para un cruce prioritario prepara la comparación en menos de 60 s y muestra los dos lados a 4 frames/s o más. Permite seguir un	Prueba de integración + demostración ante el director

Entregable	Criterio de aceptación	Herramienta de verificación
	vehículo y un peatón con HUD de velocidad, riesgo y decisiones, y el minimapa los ubica.	

8.4 Organización del Proyecto y Comunicación

8.4.1 Interfaces Externas

Tabla 8: Entidades externas e interfaces.

Entidad	Rol en el proyecto	Medio de contacto
Ing. Andrés Calderón Romero (Director)	Revisor, aprobador de entregables y asesor técnico	andrescalderonr@javeriana.edu.co / Teams
Secretaría Distrital de Movilidad	Propietaria de los datos de entrada (encuesta y aforos)	Portal de datos abiertos de Bogotá
Jurados evaluadores	Evaluación final del proyecto de grado	Coordinación académica
OpenStreetMap Community	Fuente de la red vial base del modelo y de las teselas del minimapa	openstreetmap.org

8.4.2 Roles del Equipo

La estructura organizacional del equipo se presenta en la Figura 3.

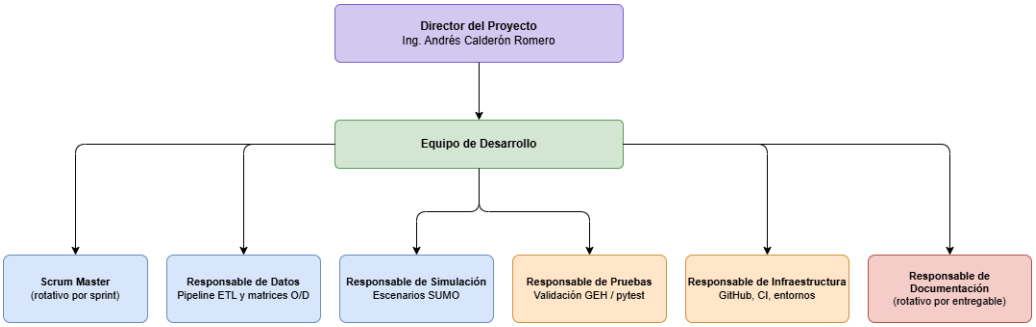


Figura 3: Organigrama del equipo de desarrollo.

Tabla 9: Roles del equipo y responsabilidades.

Rol	**Integrante(s)**	**Responsabilidades principales**
Scrum Master (rotativo por sprint)	Rotación entre integrantes	Facilitar ceremonias Scrum y actualizar burndown chart.
Responsable de datos		Liderar Fases 1 y 2 (CRISP-DM): exploración, limpieza, matrices O/D.
Responsable de simulación		Liderar Fase 3 (Scrum): construcción y calibración del escenario en SUMO.
Responsable de documentación	Rotación por entregable	Coordinar redacción y revisión de cada entregable formal.
Responsable de pruebas		Definir y ejecutar pruebas unitarias del pipeline y validación del modelo.
Responsable de infraestructura		Configurar y mantener GitHub, Actions, entornos Python y SUMO.

Scrum Master. Integrantes responsables: rotativo por sprint entre todos los integrantes. El rol de Scrum Master se implementa de manera rotativa para garantizar que todos los integrantes participen activamente en la gestión del proyecto y comprendan el estado global del desarrollo. Cada sprint tiene un Scrum Master diferente, responsable de coordinar las actividades del equipo durante ese período.

Responsabilidades principales:

- Planificar y moderar las reuniones semanales de seguimiento.
- Coordinar la reunión de planeación al inicio de cada sprint.
- Actualizar el backlog del proyecto en GitHub Projects.
- Mantener actualizado el burndown chart y las métricas de avance.
- Verificar que las tareas asignadas tengan responsables y fechas definidas.
- Identificar bloqueos técnicos u organizacionales.
- Escalar problemas críticos al director cuando sea necesario.
- Facilitar la comunicación entre los integrantes.
- Coordinar retrospectivas al cierre de cada sprint.

- Garantizar que las actividades priorizadas estén alineadas con los objetivos del sprint.

Responsabilidades de comunicación: el Scrum Master es responsable de consolidar el estado semanal del proyecto y comunicarlo durante las reuniones con el director. También debe asegurar que todas las decisiones importantes queden registradas en GitHub Issues o en la documentación formal correspondiente.

Entregables asociados: backlog actualizado, burndown charts, registro de reuniones, registro de retrospectivas y seguimiento de issues y tareas.

Responsable de Datos. Lidera las actividades relacionadas con adquisición, limpieza, análisis, transformación y validación de los datos de entrada utilizados por el sistema. Este rol tiene especial relevancia durante las Fases 1 y 2 del proyecto, correspondientes a comprensión del dominio y preparación de datos bajo CRISP-DM.

Responsabilidades principales:
- Analizar la estructura de la Encuesta de Movilidad 2023.
- Comprender y documentar la estructura de los aforos vehiculares HMD.
- Validar calidad, consistencia y cobertura de los datos.
- Diseñar el pipeline ETL de procesamiento.
- Implementar scripts de limpieza y transformación.
- Aplicar factores de expansión muestral.
- Construir matrices origen-destino.
- Verificar consistencia estadística de los resultados.
- Gestionar los datasets del proyecto.
- Documentar supuestos y limitaciones de los datos.
- Coordinar validaciones cruzadas entre aforos y matrices O/D.

Responsabilidades técnicas: el Responsable de Datos también debe garantizar que los formatos generados sean compatibles con las herramientas del ecosistema SUMO, particularmente con duarouter y netconvert. Asimismo, debe asegurar que los scripts sean reproducibles y correctamente documentados, permitiendo que cualquier integrante pueda ejecutar nuevamente el pipeline con los mismos resultados.

- Herramientas asociadas: Python, pandas, geopandas, Jupyter Notebooks, OpenStreetMap y GitHub.
- Entregables asociados: scripts ETL, matrices O/D, reportes de calidad de datos, archivos procesados de entrada y validaciones estadísticas.

Responsable de Simulación. Lidera todas las actividades relacionadas con la construcción, calibración y ejecución de escenarios dentro del simulador SUMO. Este rol tiene especial relevancia durante la Fase 3 del proyecto, correspondiente a la construcción iterativa del escenario de simulación. En la versión 2.0 incluye la definición de las zonas, del semáforo propuesto y de los indicadores de riesgo del simulador 3D.

Responsabilidades principales:
- Configurar SUMO y las herramientas complementarias.
- Procesar la red vial mediante netconvert.
- Configurar escenarios de simulación.
- Generar archivos `.rou.xml` y de tipos de vehículo.
- Integrar modos de transporte dentro del simulador.
- Ajustar parámetros de comportamiento vehicular.
- Ejecutar simulaciones de validación.
- Analizar resultados de congestión y movilidad.

- Implementar escenarios hipotéticos.
- Verificar que las simulaciones sean reproducibles.
- Documentar parámetros de calibración utilizados.

Responsabilidades de validación: el Responsable de Simulación debe verificar que las simulaciones ejecuten correctamente y que los resultados producidos mantengan coherencia con los patrones observados en los datos reales. Además, debe coordinar la validación mediante métricas GEH y RMSE junto con el Responsable de Pruebas.

- Herramientas asociadas: SUMO, netconvert, duarouter, TraCI, Python y QGIS.
- Entregables asociados: escenarios SUMO, archivos de rutas, archivos de tipos de vehículo, resultados de simulaciones, reportes de calibración y escenarios hipotéticos.

Responsable de Pruebas y Validación. Tiene como propósito garantizar la calidad técnica del sistema y verificar que los resultados producidos por el pipeline sean consistentes y correctos.

Responsabilidades principales:
- Diseñar casos de prueba del pipeline ETL.
- Diseñar pruebas unitarias para scripts críticos.
- Configurar pytest y GitHub Actions.
- Ejecutar pruebas automáticas.
- Analizar resultados de cobertura.
- Validar consistencia de archivos generados.
- Verificar métricas GEH y RMSE.
- Coordinar validación funcional del pipeline.
- Registrar defectos encontrados.
- Realizar seguimiento de errores reportados.
- Verificar corrección de errores.

Responsabilidades de calidad: este rol debe asegurar que cada incremento del proyecto cumpla con la definición de «done» establecida por el equipo. También es responsable de garantizar que los cambios realizados no introduzcan regresiones en funcionalidades previamente validadas.

- Herramientas asociadas: pytest, GitHub Actions, pylint, Python y GitHub Issues.
- Entregables asociados: casos de prueba, reportes de cobertura, registro de defectos, validaciones GEH y resultados de pruebas.

Responsable de Infraestructura. Administra los entornos de desarrollo y garantiza que el equipo disponga de las herramientas necesarias para ejecutar el proyecto de manera estable y reproducible.

Responsabilidades principales:
- Configurar el repositorio principal en GitHub.
- Definir la estructura de carpetas del proyecto.
- Configurar GitHub Actions.
- Mantener actualizados `requirements.txt` y `pyproject.toml`.
- Configurar entornos virtuales Python.
- Gestionar versiones de SUMO.
- Verificar compatibilidad entre herramientas.
- Gestionar el almacenamiento de datasets.
- Administrar permisos y ramas del repositorio.
- Verificar respaldos y versionamiento.

Responsabilidades operativas: el Responsable de Infraestructura debe apoyar a los demás integrantes cuando existan problemas de instalación, dependencias o conflictos de configuración. También debe

garantizar que la infraestructura utilizada permita la ejecución reproducible del pipeline en diferentes equipos.

- Herramientas asociadas: GitHub, Git, GitHub Actions, Python, SUMO, Node.js y Google Colab.
- Entregables asociados: repositorio configurado, integración continua funcional, entornos reproducibles y configuración de herramientas.

Responsable de Documentación. Integrantes responsables: rotativo por entregable. Coordina la elaboración, revisión y consolidación de todos los documentos formales del proyecto.

Responsabilidades principales:
- Coordinar la redacción de entregables.
- Verificar consistencia entre documentos.
- Mantener formato y estructura de estándares IEEE.
- Consolidar diagramas y tablas.
- Actualizar el historial de cambios.
- Gestionar referencias bibliográficas.
- Revisar ortografía y redacción.
- Verificar trazabilidad entre documentos.
- Coordinar revisiones internas.
- Incorporar observaciones del director.

Responsabilidades de calidad documental: el Responsable de Documentación debe garantizar que todos los documentos mantengan coherencia con el estado actual del proyecto y con las decisiones tomadas durante el desarrollo. Asimismo, debe asegurar que las versiones entregadas cumplan con los lineamientos establecidos por la Pontificia Universidad Javeriana y los estándares IEEE seleccionados.

Entregables asociados: SPMP, SRS, SDD, plan de pruebas, informe final, diagramas BPMN, tablas y anexos.

Responsabilidad compartida del equipo. Aunque existen roles especializados, el proyecto adopta un enfoque colaborativo donde todos los integrantes participan activamente en decisiones críticas relacionadas con arquitectura, calidad, alcance y validación. Las siguientes actividades son responsabilidad compartida del equipo:
- Participación en reuniones semanales.
- Revisión de código.
- Revisión de documentación.
- Discusión de decisiones técnicas.
- Identificación de riesgos.
- Resolución de problemas críticos.
- Planeación de sprints.
- Presentaciones y sustentaciones.

El equipo considera que esta distribución balanceada de responsabilidades permite reducir dependencias críticas entre integrantes y aumentar la resiliencia del proyecto frente a retrasos o dificultades técnicas.

9 Administración del Proyecto

9.1 Métodos y Herramientas de Estimación

Enfoque general de estimación. La estimación del proyecto se realizó mediante una combinación de métodos cuantitativos y cualitativos con el objetivo de obtener una planeación realista tanto del esfuerzo de desarrollo como de las actividades de documentación, validación y gestión. Debido a que

el proyecto involucra componentes de procesamiento de datos, simulación de tráfico y desarrollo de software académico, no resulta suficiente utilizar un único método de estimación tradicional. Por esta razón se adoptó una estrategia híbrida basada en:

- Estimación por analogía.
- Estimación por descomposición (WBS).
- Estimación basada en experiencia del equipo.
- Estimación incremental por sprint.
- Estimación de carga de documentación y validación.

El proceso de estimación fue realizado colectivamente por el equipo durante la planeación inicial del proyecto y refinado después de reuniones con el director.

Estimación por analogía. La primera aproximación de esfuerzo se realizó mediante comparación con proyectos similares encontrados en artículos académicos, repositorios públicos y tesis relacionadas con simulación de tráfico usando SUMO. Se analizaron proyectos que incluían:

- Procesamiento ETL de datos de movilidad.
- Construcción de matrices O/D.
- Conversión de redes OpenStreetMap.
- Integración con SUMO.
- Validación mediante métricas de tráfico.

A partir de estas referencias se identificó que un proyecto de complejidad similar normalmente requiere entre 2 y 4 meses de trabajo parcial para equipos de 3 o 4 personas. La comparación permitió establecer un rango inicial de esfuerzo aproximado y validar que el alcance propuesto era viable dentro del semestre académico.

Estimación basada en complejidad técnica. Posteriormente, el equipo clasificó cada componente del sistema según su complejidad técnica.

Tabla 10: Complejidad técnica por componente.

Componente	**Complejidad**	**Justificación**
Construcción de matrices O/D	Alta	Transformaciones estadísticas y expansión muestral
Procesamiento de red vial	Alta	Conversión OSM-SUMO y validación geoespacial
Configuración de SUMO	Media-alta	Curva de aprendizaje del simulador
Validación GEH	Media	Implementación matemática y comparación estadística
Escenarios hipotéticos	Alta	Requiere calibración y análisis comparativo
Documentación IEEE	Media	Alto volumen documental y trazabilidad
Módulo de semaforización *(v2.0)*	Media-alta	Lectura y escritura de `tlLogic`, Webster, recorte de red y demanda
Simulador 3D comparativo *(v2.0)*	Alta	Dos SUMO en lockstep por TraCI, render 3D en el navegador e indicadores de riesgo

Esta clasificación permitió priorizar tiempo adicional para las actividades de mayor riesgo técnico.

Estimación por descomposición (WBS). Después de la estimación inicial, el proyecto fue descompuesto en actividades específicas utilizando una Work Breakdown Structure (WBS). Cada actividad fue estimada individualmente considerando: complejidad técnica, dependencias, experiencia previa del

equipo, tiempo de aprendizaje requerido, riesgos asociados y actividades de pruebas y correcciones. La suma de estas estimaciones produjo el total de horas-persona del proyecto.

Herramientas utilizadas para la estimación. GitHub Projects se utilizó para estructurar el backlog inicial y dividir el trabajo en tareas concretas. Cada actividad fue registrada como issue o tarea independiente, permitiendo estimar el tiempo esperado, las dependencias, la prioridad, los responsables y el estado de avance. La herramienta también facilita el seguimiento posterior del cumplimiento de las estimaciones.

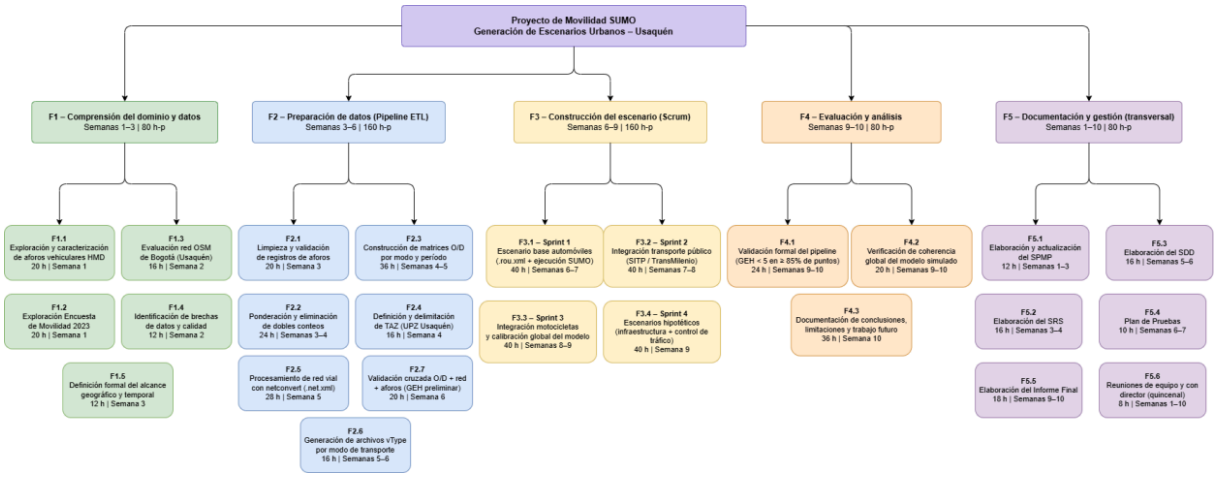


Figura 4: Diagrama WBS (línea base v1.x).

La estructura WBS (Figura 4) fue utilizada para organizar el proyecto jerárquicamente y visualizar las fases principales, las subtareas, las dependencias críticas y la relación entre entregables. Gracias a esta estructura fue posible estimar el esfuerzo de manera más precisa y detectar actividades críticas. La figura corresponde a la línea base; los paquetes de trabajo F3.5 a F3.8 de la versión 2.0 se agregan en la Tabla 15.

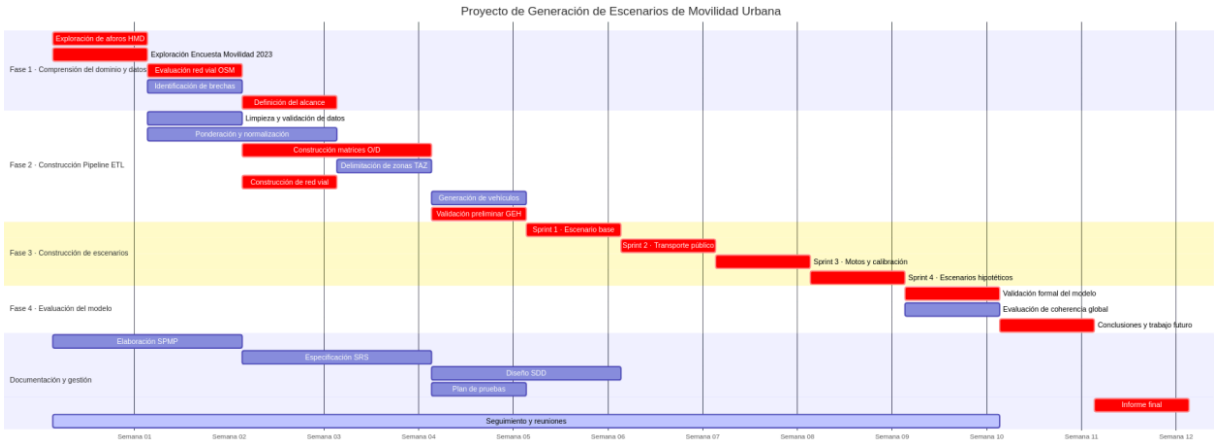


Figura 5: Diagrama Gantt (línea base v1.x).

El cronograma general se representó con un diagrama Gantt (Figura 5) para visualizar:

- la distribución temporal de actividades;
- las dependencias y la ruta crítica;
- las semanas de contingencia y las actividades paralelas.

El Gantt permitió validar que las actividades más complejas terminarían antes del cierre formal del semestre.

Estimación de horas-persona. El esfuerzo total del proyecto fue estimado utilizando horas-persona como unidad principal, considerando 4 integrantes, 10 semanas efectivas de trabajo y aproximadamente 14 horas semanales por integrante: 560 horas-persona = 4 integrantes × 10 semanas × 14 horas/semana. Esta estimación contempla tanto actividades técnicas como documentación, reuniones, pruebas y correcciones.

Tabla 11: Asignación de recursos por fase.

Fase / Componente	**H-P estimadas**	**Justificación**
Fase 1 – Comprensión del dominio y datos	80 h-p	Exploración de fuentes de datos heterogéneas con análisis de calidad y brechas.
Fase 2 – Preparación de datos (ETL)	160 h-p	Componente más complejo: limpieza, matrices O/D, red vial, validación cruzada.
Fase 3 – Construcción del escenario (Sprints 1 a 5)	240 h-p	5 sprints × ~40 h/sprint; incluye calibración, escenarios hipotéticos y desarrollo del módulo web (backend FastAPI + frontend de visualización).
Fase 3 – Extensión v2.0 (Sprints 6 a 8)	120 h-p	Semaforización, simulador 3D comparativo y pendientes (reporte de la comparación, programa editable, transporte público con GTFS).
Fase 4 – Evaluación y análisis	80 h-p	Validación formal, análisis de resultados, documentación de conclusiones.
Documentación y gestión (transversal)	80 h-p	SPMP, SRS, SDD, Plan de pruebas, Informe final, reuniones.
Total	720 h-p	Línea base de 600 h-p (560 h-p + 40 h del Sprint 5) más 120 h-p de la extensión v2.0.

Estimación de riesgo y contingencia. El equipo decidió planificar únicamente 10 semanas efectivas de trabajo dentro de un semestre de 16 semanas. Esta decisión se tomó deliberadamente para disponer de tiempo adicional destinado a:

- corrección de errores y retrasos técnicos;
- ajustes de calibración y revisión documental;
- cambios solicitados por el director;
- problemas de integración y riesgos académicos.

La estrategia busca evitar que el proyecto llegue al cierre del semestre sin margen de maniobra. La extensión v2.0 se planea dentro de ese margen (semanas 11 a 14), dejando las semanas 15 y 16 para el informe final y la sustentación.

Validación de las estimaciones. Las estimaciones fueron revisadas colectivamente y ajustadas durante reuniones con el director del proyecto. Los principales criterios utilizados para validarlas fueron:

- viabilidad dentro del tiempo disponible;
- consistencia entre alcance y recursos;
- balance de carga entre integrantes;
- riesgo técnico aceptable;
- compatibilidad con otros compromisos académicos.

Las estimaciones podrán ajustarse durante el desarrollo si aparecen riesgos o cambios importantes en el alcance.

9.2 Inicio del Proyecto

9.2.1 Entrenamiento del Personal

Tabla 12: Entrenamiento del personal.

Conocimiento / Habilidad	**Estado actual**	**Plan de entrenamiento**	**Semana**
SUMO (configuración y uso básico)	Nivel básico	Tutoriales oficiales SUMO + documentación TraCI + práctica con dataset de ejemplo.	1–2
netconvert y duarouter	Sin experiencia	Práctica guiada con la red OSM de Usaquén descargada.	2
geopandas y análisis geoespacial	Nivel básico-intermedio	Tutoriales de DataCamp + práctica con los datos de aforos y UPZ.	1–3
pytest y GitHub Actions	Nivel básico	Configuración del repositorio con el primer pipeline de CI en la semana 1.	1
TraCI en vivo y three.js *(v2.0)*	Nivel básico	Documentación de TraCI (suscripciones, `getJunctionFoes`) y ejemplos oficiales de three.js; prototipo de la escena con una zona recortada.	11

9.2.2 Infraestructura

Objetivo de la infraestructura del proyecto. La infraestructura del proyecto tiene como propósito proporcionar un entorno de desarrollo reproducible, colaborativo y estable que permita ejecutar correctamente el pipeline ETL, las simulaciones SUMO y las actividades de documentación y validación. Debido a que el proyecto involucra procesamiento de datos, simulación microscópica de tráfico y trabajo colaborativo entre varios integrantes, la infraestructura debe garantizar:

- compatibilidad entre herramientas y versionamiento adecuado;
- reproducibilidad de resultados e integración continua;
- disponibilidad de datos y colaboración eficiente.

Todas las herramientas seleccionadas son de código abierto o gratuitas para uso académico.

Tabla 13: Infraestructura y configuración.

Tarea	**Responsable**	**Semana**
Crear repositorio GitHub con estructura base (`src/`, `tests/`, `docs/`, `data/`)	Erick Salazar	1
Configurar GitHub Actions con pytest + pylint	Erick Salazar	1
Instalar SUMO en los equipos del equipo	Todos	1

Tarea	Responsable	Semana
Configurar entorno virtual Python con `requirements.txt`	Erick Salazar	1
Descargar y versionar datasets base (aforos HMD, encuesta, OSM Usaquén)	Felipe Garrido	1–2
Configurar GitHub Projects para backlog y burndown chart	Scrum Master semana 1	1
(v2.0) Instalar `simulation-service` (`pip install -e ".[dev]"`, SUMO por `eclipse-sumo`) y el visor (`npm install`, `VITE_MAPBOX_TOKEN`)	Todos	6
(v2.0) Instalar el compilador de documentos Typst (`pip install typst`)	Responsable de documentación	11

Configuración inicial del repositorio GitHub. La primera actividad de infraestructura consiste en la creación del repositorio principal del proyecto en GitHub. Tareas asociadas:
- crear el repositorio privado inicial;
- definir la estructura de carpetas y configurar las ramas principales;
- definir reglas de merge y convenciones de commits;
- crear el README inicial y configurar permisos de colaboradores.

Responsables: Responsable de infraestructura y Scrum Master del Sprint 1. Momento de ejecución: semana 1. Frecuencia: actividad inicial con mantenimiento continuo durante todo el proyecto.

Configuración del entorno Python. El proyecto utiliza Python como lenguaje principal del pipeline ETL y de integración con SUMO. Tareas asociadas:
- instalar Python 3.11+ y crear el entorno virtual;
- configurar `requirements.txt` (raíz) y `pyproject.toml` (`simulation-service`) e instalar las librerías necesarias;
- verificar compatibilidad entre dependencias y actualizarlas cuando sea necesario.

Responsables: Responsable de infraestructura y todos los integrantes. Momento de ejecución: semana 1. Frecuencia: revisión y mantenimiento semanal.

Instalación y configuración de SUMO. SUMO constituye la herramienta central de simulación del proyecto. Tareas asociadas:
- descargar la versión estable de SUMO y configurar las variables de entorno (`SUMO_HOME`);
- verificar el funcionamiento de netconvert y duarouter;
- configurar TraCI, ejecutar simulaciones de prueba y verificar la compatibilidad con Python.

Responsables: Responsable de simulación y Responsable de infraestructura. Momento de ejecución: semana 1. Frecuencia: validación continua al inicio de cada sprint.

Configuración de GitHub Actions. La integración continua permite ejecutar pruebas automáticas y mantener calidad constante del código. Tareas asociadas:
- configurar los workflows de CI e integrar pytest y pylint;
- verificar la ejecución automática y la validación por pull requests;
- corregir los errores del pipeline de CI.

Responsables: Responsable de infraestructura y Responsable de pruebas. Momento de ejecución: semana 1. Frecuencia: monitoreo continuo por commit.

Descarga y gestión de datasets. Los datasets constituyen uno de los activos más importantes del proyecto. Tareas asociadas:
- descargar la Encuesta de Movilidad, los aforos HMD y la red OpenStreetMap;
- verificar la integridad de los archivos y versionar los datasets procesados;
- mantener una estructura organizada de datos y generar respaldos locales.

Responsables: Responsable de datos y Responsable de infraestructura. Momento de ejecución: semanas 1 y 2. Frecuencia: actualización cuando existan nuevas versiones o correcciones.

Configuración del sistema de gestión de proyecto. El seguimiento de tareas se realizará mediante GitHub Projects. Tareas asociadas:
- crear el tablero Kanban y definir las columnas del flujo;
- registrar el backlog inicial y los issues por sprint;
- configurar etiquetas y milestones.

Responsables: Scrum Master y Responsable de infraestructura. Momento de ejecución: semana 1. Frecuencia: actualización diaria.

Mantenimiento de la infraestructura. La infraestructura requiere mantenimiento continuo para garantizar estabilidad y reproducibilidad. Actividades de mantenimiento:
- actualización de dependencias y verificación de compatibilidad;
- respaldo de información crítica;
- monitoreo de los pipelines de CI;
- limpieza de ramas obsoletas;
- corrección de conflictos de versiones y validación periódica de entornos.

Responsables: Responsable de Infraestructura y todos los integrantes para la validación local. Frecuencia: semanal o cuando se detecten problemas.

Tabla 14: Riesgos de infraestructura.

Riesgo	Impacto	Mitigación
Incompatibilidad de versiones	Alto	Uso de `requirements.txt` / `pyproject.toml` y documentación detallada
Fallos en GitHub Actions	Medio	Validación manual temporal
Pérdida de datos	Alto	Backups periódicos y versionamiento
Problemas de instalación de SUMO	Alto	Guías compartidas, soporte interno y paquete pip `eclipse-sumo`
Dependencias rotas	Medio	Congelamiento de versiones estables (p. ej. `traci==1.27.1`, `three@0.169.0`)

9.3 Planes de Trabajo del Proyecto

9.3.1 Calendarización con ruta crítica

La siguiente tabla presenta la descomposición completa de actividades (WBS) con la duración estimada, las semanas de ejecución, los prerrequisitos (actividades que deben estar completas antes de iniciar) y los co-requisitos (actividades que deben ejecutarse en paralelo). Las actividades resaltadas corresponden a fases de nivel superior; las marcadas *(v2.0)* son la extensión del alcance, cuyo estado al 13/10/2026 se indica en la última columna.

El camino crítico del proyecto es: F1.1 → F1.2 → F1.3 → F1.5 → F2.5 → F2.7 → F3.1 → F3.2 → F3.3 → F3.4 → F4.1 → F4.3. Cualquier retraso en estas actividades impacta directamente la fecha de

finalización del proyecto. La extensión v2.0 (F3.6 ⟶ F3.7 ⟶ F3.8) corre en paralelo a la Fase 4 y no está en el camino crítico.

Tabla 15: Calendarización con camino crítico (WBS + prerrequisitos).

ID	Actividad	H-P	Semana(s)	Prerreq.	Co-req.	Estado
F1	**FASE 1 – Comprensión del dominio y los datos**	**80 h**	**1–3**	**—**	**—**	**Terminada**
F1.1	Exploración y caracterización de aforos vehiculares HMD	20 h	1	—	F1.2	Terminada
F1.2	Exploración de la Encuesta de Movilidad 2023	20 h	1	—	F1.1	Terminada
F1.3	Evaluación de la red OSM de Bogotá para Usaquén	16 h	2	F1.1, F1.2	—	Terminada
F1.4	Identificación de brechas de datos y calidad	12 h	2	F1.1, F1.2	F1.3	Terminada
F1.5	Definición formal del alcance geográfico y temporal	12 h	3	F1.3, F1.4	—	Terminada
F2	**FASE 2 – Preparación de datos (Pipeline ETL)**	**160 h**	**3–6**	**F1**	**—**	**Terminada**
F2.1	Limpieza y validación de registros de aforos	20 h	3	F1.1	F2.2	Terminada
F2.2	Aplicación de factores de ponderación y eliminación de dobles conteos	24 h	3–4	F1.2	F2.1	Terminada
F2.3	Construcción de la demanda O/D por modo y período	36 h	4–5	F2.1, F2.2	F2.4	Terminada (ventana AM)
F2.4	Definición y delimitación de TAZ (UPZ de Usaquén)	16 h	4	F1.5	F2.3	Terminada
F2.5	Procesamiento de red vial con netconvert (`.net.xml`, red 3D con elevación)	28 h	5	F1.3	—	Terminada
F2.6	Generación de archivos vType por modo de transporte	16 h	5–6	F2.3	F2.5	Terminada
F2.7	Validación cruzada O/D + red + aforos (GEH preliminar)	20 h	6	F2.3, F2.5, F2.6	—	Pendiente (GEH)
F3	**FASE 3 – Construcción del escenario (Scrum, Sprints 1 a 8)**	**320 h**	**6–14**	**F2**	**—**	**En curso**
F3.1	Sprint 1: Escenario base (`.rou.xml` + ejecución SUMO)	40 h	6–7	F2.7	—	Terminada
F3.2	Sprint 2: Integración transporte público (SITP / TransMilenio)	40 h	7–8	F3.1	—	Parcial (como caminata; GTFS en F3.8)
F3.3	Sprint 3: Integración motocicletas y calibración global del modelo	40 h	8–9	F3.2	—	Parcial (calibración pendiente)
F3.4	Sprint 4: Escenarios hipotéticos (presets de demanda, estilos de conducción, cierres viales, control de tráfico)	40 h	9	F3.3	—	Terminada
F3.5	Sprint 5: Módulo web (API, visor kepler.gl, parametrización, precalculadas, progreso y cancelación)	40 h	9–10	F3.4	—	Terminada
F3.6	*(v2.0)* Sprint 6: Intersecciones y semaforización (ficha, programas, Webster, zonas, comparación, priorización)	40 h	11–12	F3.5	—	Terminada
F3.7	*(v2.0)* Sprint 7: Simulador 3D comparativo (semáforo propuesto, lockstep, 3ª persona, HUD, riesgo, minimapa)	40 h	12–13	F3.6	—	Terminada
F3.8	*(v2.0)* Sprint 8: Reporte de la comparación 3D, programa editable del semáforo propuesto, transporte público con GTFS	40 h	13–14	F3.7	F4.1	Pendiente

ID	Actividad	H-P	Semana(s)	Prerreq.	Co-req.	Estado
F4	**FASE 4 – Evaluación y análisis**	**80 h**	**9–14**	**F3.5**	**—**	**Pendiente**
F4.1	Validación formal del pipeline (GEH < 5 en ≥ 85% de puntos)	24 h	9–10	F3.4	F4.2	Pendiente
F4.2	Verificación de coherencia global del modelo simulado	20 h	9–10	F3.4	F4.1	Pendiente
F4.3	Documentación de conclusiones, limitaciones y trabajo futuro	36 h	10–14	F4.1, F4.2	—	Pendiente
F5	**DOCUMENTACIÓN Y GESTIÓN (transversal)**	**80 h**	**1–16**	**—**	**—**	**En curso**
F5.1	Elaboración y actualización del SPMP	12 h	1–3, 11	—	F5.2	v2.0
F5.2	Elaboración del SRS	16 h	3–4, 11	F5.1	—	v2.0
F5.3	Elaboración del SDD	16 h	5–6	F5.2	—	Pendiente
F5.4	Elaboración del Plan de Pruebas	10 h	6–7	F5.2	—	Pendiente
F5.5	Elaboración del Informe Final	18 h	14–16	F4.3	—	Pendiente
F5.6	Reuniones de equipo (lunes) y con director (quincenal)	8 h	1–16	—	Todos	En curso

9.3.2 Asignación de responsabilidades por fase

Tabla 16: Asignación de responsabilidades por fase e integrante.

Fase / Actividad	Samuel Campos	Erick Salazar	Santiago Chitiva	Felipe Garrido
Fase 1 – Dominio y datos	Líder	Co-líder	Apoyo	Apoyo
Fase 2 – Pipeline ETL	Co-líder (O/D)	Co-líder (limpieza)	Líder (red OSM)	Apoyo (validación)
Fase 3 – Escenarios (Scrum)	Apoyo	Apoyo (pruebas)	Co-líder (simulación)	Líder (calibración)
Fase 3 – Módulo Web	Apoyo	Líder	Apoyo	Apoyo
Fase 3 – Semaforización y simulador 3D *(v2.0)*	Por definir en la planeación de los Sprints 6 a 8			
Fase 4 – Evaluación	Apoyo	Líder (métricas)	Apoyo	Co-líder (análisis)
Documentación	SRS	Plan de pruebas	SDD / Informe	SPMP / Informe

10 Monitoreo y Control del Proyecto

10.1 Administración de Requerimientos

Los requerimientos del sistema se gestionan a través del SRS (entregable de la semana 4; versión 2.0 en la semana 11). El proceso de administración de requerimientos incluye: identificación inicial, revisión con el director, aprobación, y control de cambios durante el desarrollo.

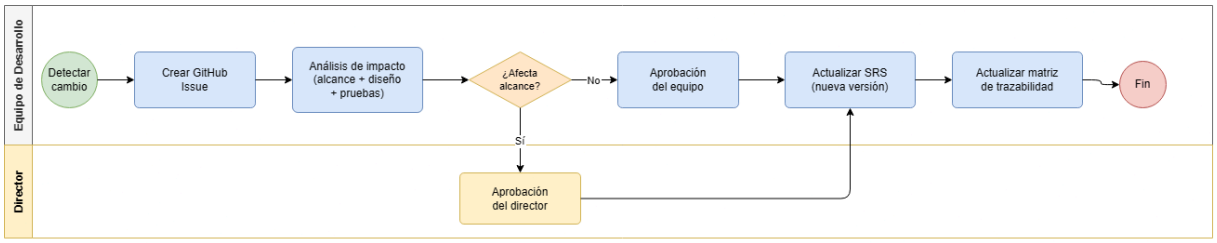


Figura 2. Diagrama BPMN – Proceso de administración de requerimientos

Figura 6: Diagrama BPMN – Proceso de administración de requerimientos.

El flujo de la Figura 6 muestra: detección del cambio → creación de GitHub Issue → análisis de impacto (alcance + diseño + pruebas) → aprobación (director si afecta alcance; equipo si es menor) → actualización del SRS con nueva versión → actualización de la matriz de trazabilidad. Roles: equipo de desarrollo, director.

Artefactos generados: SRS (requerimientos funcionales y no funcionales), matriz de trazabilidad (SRS ↔ diseño ↔ pruebas), registro de cambios de requerimientos en GitHub Issues.

Proceso de control de cambios: toda solicitud de cambio a un requerimiento aprobado debe documentarse como un GitHub Issue, ser analizada por el equipo (impacto en alcance, calendario y diseño), aprobada por el director si afecta el alcance, e incorporada al SRS con actualización de la versión. Los cambios se discuten al inicio de cada sprint. Desde la versión 2.0 cada requerimiento del SRS indica su estado (Implementado, Parcial o Pendiente), lo que facilita la métrica de requerimientos completados.

10.2 Monitoreo y Control de Progreso

El monitoreo y control del proyecto tiene como propósito verificar continuamente que el desarrollo avance de acuerdo con el alcance, cronograma y objetivos definidos en el SPMP. El proceso de seguimiento busca identificar tempranamente desviaciones, retrasos o problemas técnicos que puedan afectar la entrega del proyecto, permitiendo aplicar acciones correctivas oportunamente. El modelo de seguimiento combina prácticas de Scrum, métricas cuantitativas y validaciones técnicas específicas para proyectos de simulación de tráfico.

Métricas de Progreso. Las métricas seleccionadas permiten medir tanto el avance del software como la calidad de los resultados producidos.

Tabla 17: Métricas del proyecto.

Métrica	Unidad	Frecuencia	Responsable
Tareas completadas vs. planificadas	Issues cerrados / total del sprint	Fin de cada sprint	Scrum Master
Burndown del sprint	Horas restantes	Diario	Scrum Master
Cobertura de pruebas unitarias	% de líneas cubiertas	Por commit (CI automático)	Responsable de pruebas
Pruebas automatizadas *(v2.0)*	Pruebas pasadas /	Por commit	Responsable de pruebas

Métrica	Unidad	Frecuencia	Responsable
	total (`simulation-service/test/`)		
Porcentaje de validación con GEH	% de puntos con GEH < 5	Fin de cada sprint de simulación	Responsable de simulación y pruebas
Velocidad del equipo	Puntos de historia por sprint	Fin de cada sprint	Scrum Master
Número de requerimientos completados	Requerimientos completados / requerimientos totales	Semanal	Responsable de documentación y Scrum Master
Errores detectados y corregidos	Número de issues de error cerrados	Semanal	Responsable de pruebas

- **Tareas completadas vs. tareas planificadas.** Mide la cantidad de tareas finalizadas respecto a las comprometidas para un sprint o fase. Unidad: número de issues cerrados / total de issues planificados. Objetivo: verificar el cumplimiento del backlog comprometido. Frecuencia: al cierre de cada sprint. Responsable: Scrum Master.
- **Horas restantes del sprint (burndown).** Mide el trabajo pendiente dentro de cada sprint. Unidad: horas restantes estimadas. Objetivo: detectar retrasos tempranos y sobrecarga de trabajo. Frecuencia: seguimiento diario. Responsable: Scrum Master.
- **Cobertura de pruebas unitarias.** Mide el porcentaje de código cubierto por pruebas automatizadas. Unidad: porcentaje de líneas cubiertas. Objetivo: garantizar calidad y estabilidad del pipeline. Frecuencia: automática por commit mediante CI. Responsable: Responsable de pruebas.
- **Pruebas automatizadas (v2.0).** Número de pruebas de `simulation-service/test/` y cuántas pasan; en la versión 2.0 son 99 y pasan todas. Objetivo: detectar regresiones en el pipeline, la API y el simulador. Frecuencia: por commit. Responsable: Responsable de pruebas. Estas pruebas aún no corren en el CI (ver R10).
- **Validación GEH.** Mide el porcentaje de puntos de aforo cuya simulación alcanza valores GEH menores a 5. Unidad: porcentaje de puntos válidos. Objetivo: validar la calibración del modelo de tráfico. Frecuencia: al cierre de cada sprint de simulación. Responsable: Responsable de simulación y Responsable de pruebas.
- **Velocidad del equipo.** Mide la cantidad de trabajo completado por sprint. Unidad: puntos de historia completados. Objetivo: estimar la capacidad real del equipo. Frecuencia: fin de cada sprint. Responsable: Scrum Master.

- **Número de requerimientos implementados.** Mide la cantidad de requerimientos funcionales implementados y verificados. Unidad: requerimientos completados / requerimientos totales. Objetivo: verificar el avance funcional del sistema. Frecuencia: semanal. Responsable: Responsable de Documentación y Scrum Master.
- **Errores detectados y corregidos.** Mide la cantidad de defectos identificados y solucionados. Unidad: número de issues de error cerrados. Objetivo: monitorear estabilidad y calidad. Frecuencia: semanal. Responsable: Responsable de Pruebas.

Actividades de reporte de progreso

- **Reuniones semanales de seguimiento.** El equipo revisa el estado del proyecto:
  - tareas completadas, bloqueos y métricas;
  - actualización del backlog, ajuste de prioridades y definición de compromisos.

  Responsables: todos los integrantes. Frecuencia: dos veces por semana.
- **Reuniones con el director.** Al cierre de cada sprint se presentan avances, se revisan entregables, se discuten riesgos y se recibe retroalimentación técnica y validación de decisiones importantes. Responsables: equipo completo y director. Frecuencia: semanal según disponibilidad.
- **Actualización de GitHub Projects.** GitHub Projects actúa como mecanismo central de seguimiento: se actualiza el estado de tareas, se registran issues, bloqueos y responsables, y se registra el progreso del sprint. Responsables: Scrum Master. Frecuencia: diaria.

Verificación de implementación exitosa. Para considerar que un requerimiento fue implementado exitosamente deben cumplirse los siguientes criterios:

- el código compila y ejecuta correctamente;
- las pruebas unitarias asociadas pasan exitosamente;
- el cambio fue revisado mediante pull request;
- el resultado es consistente con el SRS;
- la funcionalidad fue validada por al menos otro integrante;
- la documentación correspondiente fue actualizada.

En el caso de componentes de simulación se requiere además la ejecución correcta en SUMO, la generación válida de archivos XML y métricas GEH dentro del rango esperado.

Acciones correctivas. Cuando las métricas indiquen desviaciones importantes respecto a la planeación inicial, el equipo aplicará acciones correctivas:

- **Retrasos en el sprint.**
  - Condición: avance inferior al 70% del trabajo comprometido.
  - Acción: reunión extraordinaria, repriorización del backlog, redistribución de tareas y reducción de alcance no crítico.
  - Responsables: Scrum Master y equipo.
- **Problemas técnicos críticos.**
  - Condición: errores que impidan continuar el desarrollo.
  - Acción: investigación colaborativa, escalamiento al director, implementación temporal alternativa y ajuste del cronograma.
  - Responsable: el responsable técnico del área afectada.
- **Baja calidad del modelo.**
  - Condición: GEH > 5 en más del 15% de puntos.
  - Acción: revisión de matrices O/D, ajuste de parámetros de simulación, validación de aforos y recalibración del escenario.
  - Responsables: Responsable de simulación y Responsable de datos.
- **Problemas de calidad del código.**
  - Condición: cobertura insuficiente o errores frecuentes.

  - Acción: incrementar pruebas, revisión obligatoria de código, refactorización y congelamiento temporal de nuevas funcionalidades.
  - Responsable: Responsable de pruebas.
- **Desviaciones documentales.**
  - Condición: inconsistencias entre entregables.
  - Acción: revisión cruzada de documentos, actualización de trazabilidad, corrección de versiones y validación colectiva.
  - Responsable: Responsable de documentación.

Cierre del monitoreo. Al finalizar cada sprint se consolidará un resumen de:
- tareas completadas y problemas encontrados;
- riesgos identificados y acciones correctivas aplicadas;
- estado de las métricas y decisiones tomadas.

Estos resultados servirán como insumo para retrospectivas y para ajustar la planeación de los siguientes sprints.

10.3 Cierre del Proyecto

Al finalizar cada entrega formal del proyecto, el equipo realiza:
- una retrospectiva de la fase, con registro de lecciones aprendidas;
- la actualización del SPMP y del registro de riesgos;
- la verificación de que todos los artefactos están versionados en GitHub.

Al cierre final (semanas 14 a 16), el equipo:
- entrega todos los artefactos al director y a la coordinación académica;
- realiza la exposición ante los jurados;
- archiva el repositorio con el código fuente y la documentación.

11 Entrega del Producto

La entrega del producto se realizará entre las semanas 10 y 16 del proyecto e incluirá los siguientes artefactos:
- Repositorio GitHub con el código fuente completo:
  - `simulation-service`: pipeline ETL, API FastAPI y simulador 3D, documentado con docstrings y README de instalación;
  - `kepler`: el visor web.
- Conjunto de archivos de configuración SUMO (`.net.xml`, `.taz.xml`, `.rou.xml`, tipos de vehículo) para la localidad de Usaquén.
- Simulaciones precalculadas de los cuatro presets de demanda (`simulation-service/precalculadas/`).
- Reporte de validación del modelo con métricas GEH y RMSE por punto de aforo.
- Resultados de los escenarios hipotéticos analizados (Sprints 3 a 7), incluidas las comparaciones 3D de semáforo propuesto en los cruces prioritarios.
- Informe final del proyecto (IEEE 1063-2001).
- Todos los entregables formales del semestre (SPMP, SRS, SDD, Plan de Pruebas) con su fuente Typst y su PDF.

La aceptación de cada entregable sigue los criterios descritos en la sección 8.3. La aceptación final está condicionada a la aprobación del informe por parte de los jurados evaluadores en la exposición de la semana 16.

12 Procesos de Soporte

12.1 Ambiente de Trabajo

El equipo opera bajo las siguientes reglas de trabajo:

- Dos reuniones semanales fijas: lunes (planeación y seguimiento, 1 hora) y viernes (revisión y retrospectiva, 30 minutos).
- Toda comunicación técnica y de decisiones relevantes debe quedar documentada en GitHub (Issues, Pull Requests, comentarios).
- El código se escribe en ramas `feat/*` y solo se integra a `develop` cuando es aprobado por al menos un compañero.
- Los documentos formales se redactan colaborativamente en Typst dentro del repositorio y se compilan a PDF para cada entrega.
- Los integrantes deben avisar con 48 horas de anticipación si no pueden asistir a una reunión o cumplir un compromiso.
- Las decisiones técnicas que afecten el diseño global se documentan como Architecture Decision Records (ADR) en el repositorio (p. ej. `docs/propuesta-semaforizacion-3d.md`).
- Siempre se priorizarán soluciones bien fundamentadas sobre soluciones rápidas.
- Siempre se priorizará la colaboración sobre el trabajo individual.
- Siempre se priorizará el impacto a largo plazo sobre los resultados a corto plazo.
- Cada integrante es responsable de sus tareas, pero puede pedir ayuda cuando lo necesite.
- Los compromisos adquiridos en reuniones deben cumplirse en los tiempos acordados.
- Se documentarán los acuerdos y decisiones importantes al finalizar cada reunión por escrito.
- Las decisiones se tomarán entre todos buscando consenso. Si no se llega a consenso, se decidirá por votación.
- Quien tenga mayor conocimiento técnico sobre un tema liderará las decisiones de esa área.
- Si una decisión resulta incorrecta, el equipo buscará una solución sin señalar ni culpar a ningún integrante.
- El feedback entre integrantes del grupo debe ser claro, respetuoso y basado en hechos concretos, nunca en ataques personales.
- Si algo incomoda o genera conflicto, debe comunicarse pronto, antes de que escale.
- Ninguna idea será descartada sin análisis previo.
- En las reuniones, todos deben tener espacio para hablar. Si alguien no participa, se le invitará respetuosamente a opinar.
- Las reuniones deben terminar con acciones concretas y un responsable asignado para cada una.
- Los errores se usan para mejorar, no para señalar a un compañero.
- Se respetarán los tiempos y espacios personales de cada integrante sin descuidar el compromiso grupal.
- Trabajar de forma aislada sin compartir avances con el equipo no está permitido.

12.2 Análisis y Administración de Riesgos

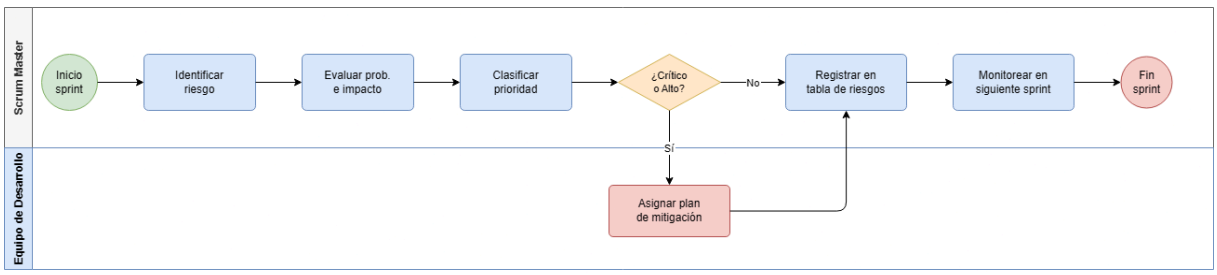


Figura 3. Diagrama BPMN – Proceso de identificación y seguimiento de riesgos

Figura 7: Diagrama BPMN – Proceso de identificación y seguimiento de riesgos.

El flujo de la Figura 7 muestra el ciclo por sprint: identificar riesgo → evaluar probabilidad e impacto → clasificar prioridad → asignar plan de mitigación → monitorear en el siguiente sprint → actualizar el registro. Roles: Scrum Master (responsable del registro), equipo de desarrollo.

Registro de Riesgos

Tabla 18: Registro de riesgos identificados.

ID	Descripción del riesgo	Prob.	Impacto	Prioridad
R1	Calidad insuficiente de los datos de la Encuesta de Movilidad para construir matrices O/D representativas.	Alta	Alto	Crítica
R2	Red vial de OSM incompleta o con errores geométricos en Usaquén.	Media	Alto	Alta
R3	SUMO no puede simular el escenario generado por el pipeline (archivos inválidos o incompatibles).	Media	Alto	Alta
R4	Estimación de tiempo insuficiente para la calibración del modelo (GEH no alcanzable en 10 semanas).	Media	Medio	Media
R5	Baja disponibilidad de un integrante del equipo por compromisos académicos o personales.	Baja	Alto	Media
R6	Datos de aforos HMD insuficientes para cubrir todos los puntos de validación de Usaquén.	Media	Medio	Media
R7	Conflictos de integración en el repositorio por trabajo simultáneo en ramas.	Alta	Bajo	Media
R8 *(v2.0)*	El simulador 3D no alcanza una animación fluida en equipos sin GPU dedicada (dos escenas 3D más dos SUMO en vivo).	Media	Medio	Media
R9 *(v2.0)*	Las conclusiones sobre semáforos se basan en programas generados por netconvert y en tráfico de fondo sintético, no en programas ni aforos reales del cruce.	Alta	Alto	Alta
R10 *(v2.0)*	Las pruebas de `simulation-service` no corren en el CI (solo las de la raíz), por lo que una regresión puede llegar a `main`.	Media	Medio	Media
R11 *(v2.0)*	Dependencia de servicios externos para los mapas base (token de Mapbox y teselas de OpenStreetMap).	Baja	Bajo	Baja

Plan de Mitigación

Tabla 19: Plan de mitigación de riesgos prioritarios.

ID	Prevención	Mitigación (si ocurre)	Responsable
R1	Exploración temprana de la encuesta en Fase 1 con análisis de calidad y cobertura.	Reducir el alcance a las UPZ con mayor cobertura muestral.	Samuel / Erick
R2	Inspección manual de la red OSM en QGIS antes de iniciar Fase 2.	Corrección de geometrías con JOSM o parámetros de netconvert.	Santiago

ID	Prevención	Mitigación (si ocurre)	Responsable
R3	Pruebas de integración del pipeline contra SUMO desde el Sprint 1.	Depuración de archivos XML con herramientas de validación de SUMO.	Felipe / Santiago
R4	Definir GEH como criterio aspiracional (no bloqueante) en sprints intermedios.	Ajustar el criterio de «done» o reducir modos integrados en acuerdo con el director.	Director + equipo
R5	Documentación continua del conocimiento en el repositorio (README, docstrings).	Redistribuir tareas entre los tres integrantes restantes.	Scrum Master
R8	Fusionar las geometrías estáticas por material, limitar la resolución del render y recortar la zona a 350 m.	Cámara aérea con menos detalle o una sola vista a la vez.	Responsable de simulación
R9	Mostrar en el visor la fuente de la demanda (real / sintética) y el origen del programa; usar viajes reales recortados cuando existan.	Calibrar el tráfico de fondo con aforos del cruce y ajustar el programa con Webster (RF-042).	Responsable de simulación + datos
R10	Ejecutar `pytest` de `simulation-service` antes de cada pull request.	Agregar el job de `simulation-service` al workflow de GitHub Actions.	Responsable de pruebas
R11	Documentar el token en `kepler/.env.example`.	Cambiar a otro proveedor de teselas en `Minimapa.jsx` / kepler.gl.	Responsable de infraestructura

12.3 Administración de Configuración y Documentación

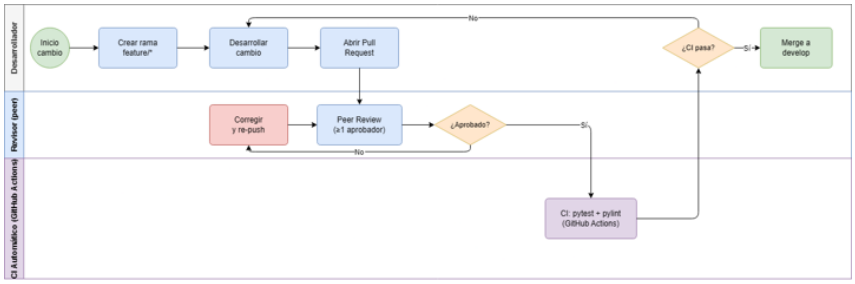


Figura 8: Diagrama BPMN – Proceso de control de configuración.

El flujo de la Figura 8 muestra:

1. cambio en código o documento;
2. creación de una rama `feat/*` y desarrollo;
3. pull request y revisión por pares (al menos un aprobador);
4. ejecución automática del CI (GitHub Actions);
5. merge a `develop`, validación final y release a `main`.

Roles: desarrollador, revisor y CI automático.

Ítems de Configuración

Tabla 20: Ítems de configuración.

Ítem de configuración	Tipo	Ubicación en el repositorio
Pipeline ETL y pasos de SUMO	Software	`simulation-service/src/etl/` (`etl.py`, `sumo/*.py`)
API (REST + WebSocket) y gestor de corridas	Software	`simulation-service/src/api/`
Visor web y simulador 3D	Software	`kepler/src/` (`components/simulador3d/`)
Pruebas unitarias e integración (pytest)	Software	`simulation-service/test/` y `tests/` (raíz)
Escenario por defecto (red 3D, TAZ, TSV, plantilla `.sumocfg`)	Datos / configuración	`simulation-service/src/etl/sumo/scenarios/default/`
Simulaciones precalculadas	Artefacto versionado	`simulation-service/precalculadas/<preset>/`
Corridas, zonas y comparaciones 3D	Artefacto generado (no versionado)	`simulation-service/runs/`
SPMP	Documento (Typst + PDF)	`docs/spmp/`
SRS	Documento (Typst + PDF)	`docs/srs/`
VFP (propuesta)	Documento (Typst + PDF)	`docs/vfp/`
SDD	Documento	`docs/sdd/`
Plan de Pruebas	Documento	`docs/testing/`
Informe Final	Documento	`docs/informe-final/`
Datos de entrada (aforos, encuesta, OSM)	Datos	`data/raw/` (Google Drive; no versionados en Git)
Red, TAZ, edificios y artefactos de los notebooks	Artefacto generado	`data/processed/`
Notebooks CRISP-DM	Software	`notebooks/` (`F1.x`, `F2.x`)
Dependencias	Configuración	`requirements.txt` (raíz), `simulation-service/pyproject.toml`, `kepler/package.json`
Configuración de GitHub Actions (`.yml`)	Configuración CI	`.github/workflows/`

Ciclo de Vida de Artefactos

Tabla 21: Ciclo de vida de artefactos.

Artefacto	Se crea en	Se actualiza en
SPMP	Semana 1–3	Cada entrega formal del proyecto (v2.0: semana 11)

Artefacto	Se crea en	Se actualiza en
SRS	Semana 3–4	Si hay cambios de requerimientos aprobados (v2.0: semana 11)
SDD	Semana 5–6	Con cambios de diseño durante Fase 3
Plan de Pruebas	Semana 6–7	Con cambios de diseño o SRS
Código fuente	Desde semana 1	Continuo (por commit)
Simulaciones precalculadas	Sprint 5	Cuando cambian la red, la TAZ, la encuesta o la plantilla `.sumocfg` (su `meta.json` lo detecta)
Informe Final	Semana 9	Semanas 9–16
Registro de riesgos	Semana 1	Inicio de cada sprint

12.4 Métricas y Proceso de Medición

Tabla 22: Métricas y proceso de medición.

Métrica	Sección de uso	Quién recolecta	Cuándo	Herramienta
Tareas completadas / sprint	10.2	Scrum Master	Fin de sprint	GitHub Projects
Burndown del sprint	10.2	Scrum Master	Diario	GitHub Projects
Cobertura de pruebas (%)	10.2	CI automático	Por commit	pytest--cov + GitHub Actions
Pruebas automatizadas que pasan	10.2	Responsable de pruebas	Por commit	pytest (`simulation-service/test/`)
GEH por punto de aforo	8.3 / 10.2	Responsable de simulación	Fin de sprint de simulación	Script Python + salida SUMO
RMSE del modelo	8.3 / 10.2	Responsable de simulación	Fin de Fase 3 y Fase 4	Script Python
Desempeño del simulador 3D (frames/s, tiempo de preparación)	8.3	Responsable de simulación	Fin del Sprint 7 y 8	Script de integración por WebSocket + perfil del navegador
Velocidad del equipo	10.2	Scrum Master	Fin de sprint	GitHub Projects

12.5 Control de Calidad

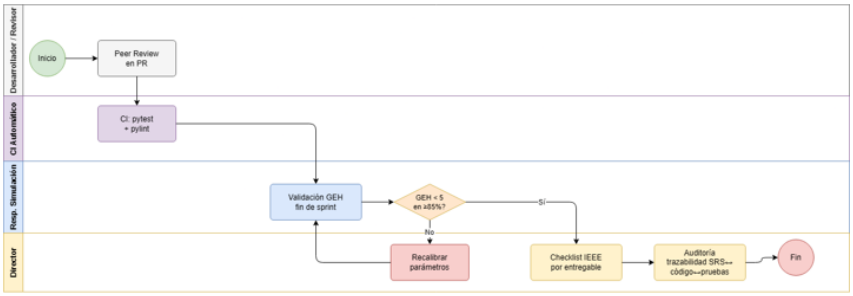


Figura 9: Diagrama BPMN – Proceso de control de calidad.

La Figura 9 muestra los tres niveles de calidad:

1. **código:** revisión por pares en cada PR y CI automático (pytest + pylint);
2. **modelo de simulación:** validación GEH al cierre de cada sprint de simulación;
3. **documentos formales:** revisión con checklist por estándar antes de cada entrega y auditoría de trazabilidad SRS ↔ código ↔ pruebas.

Roles: desarrollador, revisor, director y CI automático.

Tabla 23: Procesos de control de calidad.

Proceso de calidad	**Momento**	**Responsable**	**Herramienta**
Revisión de cambios (peer review)	Por cada PR	Al menos 1 compañero	GitHub PR reviews
Pruebas unitarias automatizadas (CI)	Por cada commit	GitHub Actions	pytest + pylint
Compilación del visor	Por cada PR que toque `kepler/`	Autor del cambio	`npm run build`
Validación GEH del modelo	Fin de cada sprint de simulación	Responsable de simulación	Script Python + aforos reales
Revisión formal de documentos	Antes de cada entrega formal	Equipo + director	Checklist por estándar IEEE + compilación Typst
Retrospectiva de sprint	Fin de cada sprint	Scrum Master	Registro de lecciones aprendidas
Auditoría de trazabilidad SRS ↔ código ↔ pruebas	Antes de cada entrega formal	Responsable de documentación	Matriz de trazabilidad (columna Estado del SRS)

13 Anexos

13.1 Anexo A – Estructura del Repositorio

Tabla 24: Estructura del repositorio.

Carpeta / Archivo	Contenido
`simulation-service/src/etl/`	Pipeline ETL: orquestador `etl.py`, simulaciones precalculadas y módulos de cada paso en `sumo/` (viajes, demanda, duarouter, simulación, exportación, intersecciones, programas, recorte, priorización, semáforo propuesto, escena 3D, telemetría)
`simulation-service/src/api/`	API FastAPI (`api.py`), gestor de corridas (`jobs.py`) y simulador 3D comparativo (`comparacion3d.py`)
`simulation-service/test/`	Pruebas pytest del servicio (99 en v2.0)
`simulation-service/precalculadas/`	Simulaciones precalculadas de los presets (versionadas)
`simulation-service/runs/`	Corridas, zonas y comparaciones generadas desde la API (no versionadas)
`kepler/`	Visor web React + kepler.gl + three.js (`src/components/simulador3d/`)
`tests/`	Pruebas de la raíz (las que corre el CI)
`data/raw/`	Datos de entrada originales (enlace a Google Drive; no versionados en Git)
`data/processed/`	Red OSM y 3D, TAZ, edificios y demás artefactos de los notebooks
`docs/`	Documentos formales en Typst y PDF (`spmp/`, `srs/`, `vfp/`) y notas de diseño
`notebooks/`	Jupyter Notebooks de exploración y preparación de datos (Fases 1 y 2)
`requirements.txt`	Dependencias Python del espacio de exploración
`.github/workflows/`	Configuración de GitHub Actions (CI/CD)
`README.md`	Guía de instalación y uso del sistema

13.2 Anexo B – Registro de Riesgos Actualizable

El registro de riesgos se mantiene actualizado en la sección 12.2 de este documento y como archivo en `docs/riesgos/` del repositorio. Se revisa al inicio de cada sprint.

13.3 Anexo C – Compilación de los documentos

Los documentos formales se compilan con Typst desde la carpeta de cada uno:

```
typst compile docs/spmp/spmp.typ                    (o python -c "import typst;
typst.compile('spmp.typ')" con pip install typst)
```

14. Referencias

[1] P. A. Lopez, M. Behrisch, L. Bieker-Walz, y J. Erdmann, «Microscopic Traffic Simulation using SUMO», en *21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575-2582.

[2] IEEE, «IEEE Std 1058-1998 – IEEE Standard for Software Project Management Plans», 1998.

[3] ISO/IEC, «ISO/IEC 12207:2008 – Systems and software engineering – Software life cycle processes», 2008.

[4] Object Management Group, «Business Process Model and Notation (BPMN) Version 2.0», 2011.

[5] Secretaría Distrital de Movilidad de Bogotá, «Encuesta de Movilidad de Bogotá 2023», Alcaldía Mayor de Bogotá, 2023.

[6] J. de D. Ortúzar y L. G. Willumsen, *Modelling Transport*, 4.ª ed. Wiley, 2011.

[7] IEEE, «IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications», 1998.

[8] IEEE, «IEEE Std 1016-2009 – IEEE Standard for Information Technology – Systems Design – Software Design Descriptions», 2009.

[9] ISO/IEC/IEEE, «ISO/IEC/IEEE 29119-1:2022 – Software and systems engineering – Software testing», 2022.

[10] F. V. Webster, «Traffic Signal Settings (Road Research Technical Paper No. 39)», Her Majesty's Stationery Office, 1958.

[11] D. Gettman, L. Pu, T. Sayed, y S. Shelby, «Surrogate Safety Assessment Model and Validation (FHWA-HRT-08-051)», Federal Highway Administration, 2008.

[12] R. Wirth y J. Hipp, «CRISP-DM: Towards a standard process model for data mining», 2000.

[13] K. Schwaber y J. Sutherland, *The Scrum Guide*. 2020.